

Tutorial and Practical work n° 3

Level: 1st Year of the Foundation training

Material: Algorithms and Static Data Structures

Exercise 01:

Write a procedure, **Exchange(a, b, c)**, that takes as input three integer numbers then exchanges their values according to their sorting order:

Example:

- Exchange(5, 7, 3) (a=5, b=7, c=3) → (a=3, b=5, c=7)

Exercise 02:

Write the following procedures:

- Read an integer number.
- Calculate the sum of two integer numbers.
- Calculate the sum of all positive integer numbers that are less than or equal to a number n ($n \leq 10$), where the sum is carried out two by two in symmetric manner while displaying the operation in each time.

Example:

- for $n = 5$, $\text{sum} = \{(1+5)+(2+4)+3\}$

Exercise 03:

Write an algorithm that calculates the scalar distance between two points $p_1(x_1, y_1)$ and $p_2(x_2, y_2)$. Use a procedure to read the coordinates, a function to calculate the distance, and finally, a procedure to display the results.

Exercise 04:

Write an algorithm to calculate $\text{LCD} = (a*b)/\text{GCD}(a, b)$; where a and b represent non-null integer numbers.

Exercise 05:

The integer part of applying square root on a number n is the sum of the first odd numbers. Write a function, **getIntegerPart(n)**, that returns the integer part of a square root.

Write a function, **verifySquare(n)**, based on the previous function which verifies if an integer number is a square number (square root gives exactly an integer)

Example:

- $\sqrt{16} = \sqrt{1 + 3 + 5 + 7} = 4$, which is the number of the first odd numbers.
- $\sqrt{15} = \sqrt{1 + 3 + 5 + 6} = 3$
- $\sqrt{25} = \sqrt{1 + 3 + 5 + 7 + 9} = 5$
- verifySquare(25) gives TRUE.
- verifySquare(15) gives FALSE.

Exercise 06:

Given a positive integer number n , write an algorithm, **longestPal(n)**, that returns the longest palindrome size.

Example:

- $n=3421267$, the longest palindrome size is 3.
- $n=1233453$, the longest palindrome size is 2.

Exercise 07

Write an algorithm that rearranges the digits of a positive integer n to form the **largest possible number** and deal with this number as a **base password**.

Write an algorithm to evaluate the **entropy** of the generated number by analyzing the count of unique digits it contains using the following formula.

Tutorial and Practical work n° 3

Level: 1st Year of the Foundation training

Material: Algorithms and Static Data Structures

$$Entropy = \log_2(\text{number of unique digits})$$

The entropy value indicates the security level of the password as follows:

- **Weak**, if $Entropy \leq 2$.
- **Strong**, otherwise.

Exercise 08:

Write an algorithm that applies a **transformation** by reversing the digits of the **base password** (generated previously) and then **subtracting** the **smallest possible number** (obtained by rearranging the digits of **n** in the smallest number) from the **base password**.

Example

- **Input:** $n = 317$
- **Base password:** 731
- **Smallest number:** 137
- **Subtract:** $731 - 137 = 594$ (This is the final **obfuscated password**)

Exercise 09:

Write an algorithm that calculates the sum of the digits raised to the power of their index (index starts from 1). This value will serve as an **encryption key** for further transformations.

After that, shuffle the digits of **n** based on the encryption key (by rotating the digits left by a number of positions equal to the last digit of the encryption key).

Example:

$$n = 442775$$

$$\begin{aligned} \text{Encryption key} &= 4^1 + 4^2 + 2^3 + 7^4 + 7^5 + 5^6 \\ &= 4 + 16 + 8 + 2401 + 16807 + 15625 \\ &= 19901 \end{aligned}$$

Use the last digit (i.e., 1) of the *Encryption key* to rotate the digits of **n** by 1 position
Shuffled number = 427754

Exercise 10:

Write an algorithm that transforms a numeric password (positive integer **n**) into a more secure representation by analyzing **consecutive repetitions** of its digits.

Example

- $n = 44455522$
- $Output = 435322$

NB:

- You can use the two predefined functions **pow(x, p)** to calculate x^p and **log2(n)** to calculate $\log_2(n)$.
- Write the main algorithm for all exercises.